Memory-Aware Agent Edge Serving

Phase 1: Baseline and Bottleneck Characterization

Jiawei Liu, Kaiqi Han, Tianyu Xu, You Chen, Yufan Shi
CSE 291A, Fall 2026

I. Problem and Setup

ReAct agents [2] alternate LLM calls with tool calls; while a tool runs, the agent's KV cache stays resident and its context grows with every step. Phase 1 asks which resource limits completed work when such agents share one GPU under a bounded KV budget: (a) KV capacity for active requests, (b) KV held by agents idle in tool calls, (c) re-prefill after the runtime evicts a context, or (d) the GPU itself when memory does not bind.

We serve Qwen2.5-7B-Instruct [4] (Q4_K_M, f16 KV) with `llama-server` [3] b11960 on an NVIDIA A10G. The KV budget is the unified pool shared by all slots (`-c`). The baseline admits tasks in FIFO order to K workers; worker k pins slot k, so an agent's KV stays resident through its tool calls (always retain). We log per-request tokens and timings, tool waits, and per-task latency, queueing, and EM/F1, and reconstruct per-slot KV occupancy, attributed to agents in a request, idle in a tool call, or finished. Throughput is measured between the 10th and 90th percentile completion. In total we ran 29 configurations over 5,890 tasks. A sweep driver restarts the server for every run, and each run records the server command line, build, workload hash, and code revision; workloads, sweep definitions, and analysis scripts are in the project repository.

II. Workload

Agents answer HotpotQA multi-hop questions [1] by searching an offline corpus of 66,581 paragraphs. HotpotQA annotates supporting facts, so every retrieved sentence can be labeled as evidence or not. The prompt grows append-only, so each step's prompt is a prefix of the next. Tool latency is lognormal (median 0.8 s) and seeded per task, step, and tool, so all configurations replay identical delays. Workloads are frozen in advance (closed-loop or Poisson arrivals) and drawn from a 1,500-question split disjoint from the development and test sets. Table I defines the four profiles. The agent reaches an F1 of 0.64 on the development set, versus 0.25 for closed-book answering.

III. Runtime Characterization

Table II lists runtime behavior and costs measured with dedicated probes and microbenchmarks. Three properties shape the results below. Over-committing the pool with active requests fails all of them at once. Under idle-slot pressure the runtime evicts by slot index, regardless of recency or size. Offloading a slot to host memory is cheaper than re-prefilling it but stalls every other slot while it runs. A cost model written from spec-sheet bandwidth and FLOPs before these measurements predicted 2.4k tok/s prefill and an 11 ms offload round trip for 2k tokens; the measured values are 4.5k tok/s and 113 ms, because slot save goes through a file on the server's main loop.

TABLE I
Task Profiles (A10G, 600 Tasks at $K = 8$)

Profile	Share	LLM calls	Tool calls	Context (tokens) first / peak p50 / p95	Output tokens	F1
`react_short`	40%	4.0	3.0	1.5k / 1.8k / 2.5k	156	0.55
`react_long`	30%	3.9	2.9	1.5k / 3.5k / 6.2k	152	0.57
`react_explain`	20%	4.9	2.9	1.7k / 2.3k / 3.1k	443	0.60
`oneshot_long`	10%	1.0	0	3.3k / 3.3k / 3.9k	44	0.50

Share is the default mix; the budget sweep (E2) uses 70% `react_long`, 30% `react_short`.

TABLE II
Runtime Behavior and Costs (llama.cpp b11960, A10G)

Probe	Observation
Active requests exceed the pool	all in-flight requests fail (HTTP 500)
Idle slots exceed the pool	lowest-index idle slot purged, one at a time
Same 1.5k-token prefix on 8 slots	stored once per slot, not shared
Decode, 1 → 64 requests (separate KV)	95 → 1,537 tok/s total
Prefill at batch 16, separate vs. unified KV	4,578 vs. 2,776 tok/s
Re-prefill, 2k / 8k tokens	0.45 / 1.97 s
Slot save / restore to host RAM	1.35 / 4.74 GB/s; blocks all slots (119 ms for 3k tokens)
`-np` 4 vs. 64 at equal `-c`	no change in GPU memory or speed

IV. Results

A. Concurrency and the Unified KV Pool (E1)

Table III and Fig. 1a show that throughput peaks at $K = 8$ and falls by 31% at $K = 32$, while tail service time grows from 22.6 s to 141.7 s (p95). Work per task is constant, but the median time per generated token rises from 15 ms at $K = 4$ to 254 ms at $K = 32$, against 28 ms for 32 decodes with separate KV buffers. In the unified pool, attention cost appears to scale with the tokens of all slots (Table II). A fixed KV region per slot removes most of the decline: 43.0, 54.8, and 64.5 tasks/min at $K = 8$, 16, and 32, i.e., +27% at $K = 16$ with the same 131k-token budget and 2.1× at $K = 32$ with twice the pool, at the cost of static reservation. With Poisson arrivals at $K = 8$ and 21, 33, and 45 tasks/min (up to 100% of capacity), all tasks complete; p95 queue wait is 0, 4.1, and 13.8 s and p99 latency 19.9, 23.5, and 33.8 s.

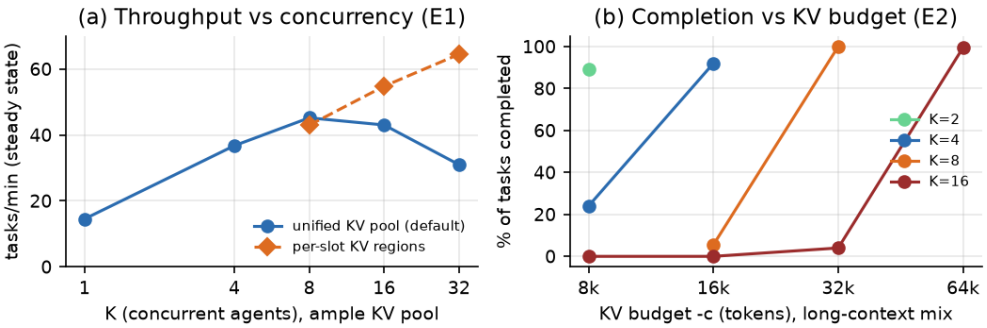


Fig. 1. (a) Steady-state throughput versus concurrency K with an ample KV pool (131k tokens), for llama.cpp's unified pool and for per-slot KV regions. (b) Fraction of tasks completed as the KV budget shrinks, long-context mix.

TABLE III
CONCURRENCY SWEEP WITH AN AMPLE KV POOL (E1)

K	Done	Tasks/min (correct)	Service time (s) p50 / p95 / p99	TTFT p50 (s)	Mean KV (idle)
1	50/50	14.5 (6.7)	3.8 / 7.8 / 10.6	0.04	2.0k (40%)
4	200/200	36.8 (17.8)	5.5 / 14.1 / 17.3	0.07	8.5k (29%)
8	600/600	**45.2** (20.6)	8.4 / 22.6 / 30.6	0.13	17.4k (18%)
16	599/600	43.0 (19.5)	17.3 / 49.2 / 62.5	0.56	34.4k (8%)
32	319/320	31.0 (15.2)	44.4 / 141.7 / 157.2	1.24	68.9k (3%)

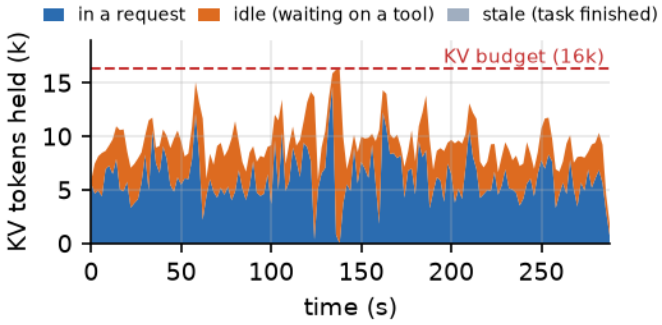


Fig. 2. KV held over time under a tight budget (16k tokens, $K = 4$, long-context mix), by owner state. The pool averages 58% occupancy, 36% of it held by agents waiting on tools. All 16 failures occur in four bursts of four, when held KV approaches the budget.

B. KV Budget and Context Growth (E2)

When the active working set exceeds the pool, every in-flight request fails and the workers' next tasks fail likewise: doubling K beyond the budget reduces completion from 89–100% to at most 24% (Fig. 1b). Contexts grow 2.3× (median) to 4.2× (p95) during a `react_long` task (Table I), so a fixed K must be provisioned for peak demand. The largest working K is about the budget divided by 4k tokens (2, 4, 8, 16 for 8k–64k), and the 8k and 16k budgets reach only 48% and 80% of the throughput of larger ones.

C. Idle KV, Retained Context, and Composition (E3–E5)

With an ample pool, scaling tool delays by 0, 1, and 4 leaves 0%, 8%, and 33% of held KV idle while throughput changes only from 44.1 to 41.0 tasks/min. At the small K that tight budgets impose, 36–44% of held KV (about a quarter of the pool) belongs to agents waiting on tools. Fig. 2 shows both sources of slack: headroom kept for peaks, and idle KV. Disabling prefix reuse increases prefill work 4.9× and cuts throughput to 36%; when llama.cpp assigns slots itself, 31% of returning steps land on another slot, costing 17%. The 1.4k-token ReAct prefix is 73% of a `react_short` context and is duplicated per slot, and only 4.5% of retrieved tokens in `react_long` contexts are supporting facts.

D. Memory, Failures, and Quality

GPU memory is fixed by the preallocated pool (5.0 GiB at 8k to 11.6 GiB at 131k tokens; server RSS $\leq$1.1 GiB), so occupied KV tokens are the varying memory metric. No request or task timed out. Of 5,890 tasks, 971 failed from context overflow (all in E2) and 9 from a client keep-alive race. F1 stays within 0.52–0.64 in every configuration that completes at least 85% of its tasks.

V. DOMINANT BOTTLENECK AND PHASE 2 DIRECTION

With ample memory, throughput is limited by attention cost in the unified pool (d), which caps useful concurrency at $K \approx 8$. Under a tight budget, active KV capacity (a) binds: contexts grow and overflow fails all in-flight work, so a fixed K is provisioned for peaks and the GPU is underutilized. Idle KV (b) is a large share of the pool only in this regime, and re-prefill after eviction (c) was rare under always-retain (at most 12 live-agent evictions per run) but costly.

Phase 2 therefore targets the memory-constrained regime with an agent-level KV lifecycle manager at tool-call boundaries and admission that reserves each agent's projected peak KV. Unlike InferCept [5], built on vLLM [6], it must use llama.cpp's slot controls, whose offload path blocks all slots. Because releasing KV costs more than retaining it through a typical tool call (break-even 1.7–3.0 s at $K = 16$ versus a 0.8 s median delay), the policy retains by default and releases the agent expected to return last only under pressure. Baselines are the best fixed K per budget, per-slot KV regions, and the runtime's default policy.

APPENDIX
TIMELINE AND MILESTONES

Dates	Work	Milestone	Status
10/21–10/28	Design; data, corpus, splits; tools, ReAct agent, profiles; workload generator; FIFO runner and instrumentation	End-to-end workload on 7B	Done
10/29–10/31	AWS A10G setup; cost model; runtime probes; microbenchmarks; experiments E1–E5	Measurements and bottleneck analysis	Done
11/1–11/4	Phase 1 report	**Phase 1 due 11/4**	Done
11/5–11/11	KV manager (slot save/restore/erase); peak-KV admission; baselines including per-slot KV	All baselines run end to end	Planned
11/12–11/18	Pressure-gated retention policy; tuning on dev; idle-KV cost test	Main comparison on dev	Planned
11/19–12/2	Arrival-rate, mix and no-benefit sweeps on test; admission and lifecycle ablation; overhead and fairness	All Phase 2 results	Planned
12/3–12/9	Final report and reproducibility package	**Final submission**	Planned

REFERENCES

[1] Z. Yang *et al.*, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. EMNLP*, 2018.
[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
[3] G. Gerganov *et al.*, "llama.cpp," https://github.com/ggml-org/llama.cpp.
[4] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.
[5] R. Abhyankar *et al.*, "InferCept: Efficient intercept support for augmented large language model inference," in *Proc. ICML*, 2024.
[6] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.